Date : 27-03-2026

Media Query fundamentals:

The **CSS Media Query** gives you a way to apply CSS only when the browser and device environment matches a rule that you specify, for example "viewport is wider than 480 pixels"

Media queries are a key part of responsive web design, as they allow you to create different layouts depending on the size of the viewport, but they can also be used to detect other things about the environment your site is running on, for example whether the user is using a touchscreen rather than a mouse.

Media query basics:

```
@media media-type and (media-feature-rule) {
  /* CSS rules go here */
}
```

- A media type, which tells the browser what kind of media this code is for (print or screen).
- A media expression, which is a rule, or test that must be passed for the contained CSS to be applied.
- A set of CSS rules that will be applied if the test passes and the media type is correct.

Media types:

The possible types of media you can specify are:

- all
- print
- screen

Media feature rules:

After specifying the type, you can then target a media feature with a rule. The following examples show how to use different media queries. To change the width of your screen, change the size of your browser or rotate your handheld device.

Weight and height:

The feature we tend to detect most often in order to create responsive designs (and that has widespread browser support) is viewport width, and we can apply CSS if the viewport is above or below a certain width — or an exact width — using the width media feature, and prefix it with min- or max- as needed.

These features are used to create layouts that respond to different screen sizes.

```
@media screen and (width: 600px) {
  body {
    color: red;
```

```
  }
}
```

The width (and height) media features can be used as ranges, and therefore be prefixed with min- or max- to indicate that the given value is a minimum, or a maximum. For example, to make the color blue if the viewport is 600 pixels or narrower, we can use max-width

```
@media screen and (max-width: 600px) {
  body {
    color: blue;
  }
}
```

Orientation:

One well-supported media feature is orientation, which allows us to test for portrait or landscape orientation.

```
@media (orientation: landscape) {
  body {
    color: rebeccapurple;
  }
}
```

Use of pointing devices:

As part of the Level 4 specification, the hover media feature was introduced. This feature means you can test if the user has the ability to hover over an element, which essentially means they are using some kind of pointing device; touchscreen and keyboard navigation does not hover.

```
@media screen and (hover: hover) {
  body:hover {
    color: white;
    background: black;
  }
}
```

Also in Level 4 is the pointer media feature. This takes three possible values, none, fine and coarse. A **fine** pointer is something like a mouse or trackpad. It enables the user to precisely target a small area.

A **coarse** pointer is your finger on a touchscreen.

The value **none** means the user has no pointing device; perhaps they are navigating with the keyboard only or with voice commands.

Using pointer can help you to design better interfaces that respond to the type of interaction a user is having with a screen. For example, you could create larger hit areas if you know that the user is interacting with the device as a touchscreen.

Using ranged syntax:

One common case is to check if the viewport width is between two values:

```
@media (min-width: 30em) and (max-width: 50em) {
  /* … */
}
```

If you want to improve the readability of this, you can use "range" syntax:

```
@media (30em <= width <= 50em) {
  /* … */
}
```

More complex media queries:

With all of the different possible media queries, you may want to combine them, or create lists of queries — any of which could be matched.

‘and’ logic in media queries:

To combine media features you can use and in much the same way as we have used and above to combine a media type and feature. For example, we might want to test for width and orientation. The body text will only be blue if the viewport is at least 600 pixels wide and the device is in landscape mode.

‘or’ logic in media queries:

If you have a set of queries, any of which could match, then you can comma separate these queries.

```
@media screen and (width >= 600px), screen and (orientation: landscape) {
  body {
    color: blue;
  }
}
```

‘not’ in media queries:

You can negate an entire media query by using the not operator. This reverses the meaning of the entire media query.

```
@media not (width >= 600px) {
  body {
    color: blue;
```

```
  }
}
```

You can also use not to negate specific expressions.

```
@media (not (width < 600px)) and (not (width > 1000px)) {
  body {
    color: blue;
  }
}
```

How to choose breakpoints:

The points at which a media query is introduced are known as **breakpoints**.

Mobile-first responsive design:

Broadly, you can take two approaches to responsive design. You can start with your desktop or widest view and then add breakpoints to move things around as the viewport becomes smaller, or you can start with the smallest view and add layout as the viewport becomes larger. This second approach is described as **mobile first** responsive design and is quite often the best approach to follow.

Viewport meta:

```
<meta name="viewport" content="width=device-width,initial-scale=1" />
```

This is the viewport meta tag — it exists as a way to control how mobile browsers render content, making sure they respect your media queries. The one above tells mobile browsers "don't render the content with a 980-pixel viewport — render it using the real device width instead, and set a default initial scale level for better consistency." The media queries will then kick in as expected.

Flexbox and CSS Grid give you ways to create flexible and even responsive components without the need for a media query: It's always worth considering whether you really need one.

JavaScript

JavaScript was initially created to “make web pages alive”.

The programs in this language are called *scripts*. They can be written right in a web page’s HTML and run automatically as the page loads.

Scripts are provided and executed as plain text. They don’t need special preparation or compilation to run.

In this aspect, JavaScript is very different from another language called Java.

Why is it called javaScript?

When JavaScript was created, it initially had another name: “LiveScript”. But Java was very popular at that time, so it was decided that positioning a new language as a “younger brother” of Java would help.

But as it evolved, JavaScript became a fully independent language with its own specification called ECMAScript, and now it has no relation to Java at all.

Today, JavaScript can execute not only in the browser, but also on the server, or actually on any device that has a special program called the JavaScript engine.

The browser has an embedded engine sometimes called a "JavaScript virtual machine".

Different engines have different "codenames". For example:

- V8 – in Chrome, Opera and Edge.
- SpiderMonkey – in Firefox.
- There are other codenames like "Chakra" for IE, "JavaScriptCore", "Nitro" and "SquirrelFish" for Safari, etc.

How the engines works?

1. The engine (embedded if it's a browser) reads ("parses") the script.
2. Then it converts ("compiles") the script to machine code.
3. And then the machine code runs, pretty fast.

The engine applies optimizations at each step of the process. It even watches the compiled script as it runs, analyzes the data that flows through it, and further optimizes the machine code based on that knowledge.

In-browser javascript:

Modern JavaScript is a "safe" programming language. It does not provide low-level access to memory or the CPU, because it was initially created for browsers which do not require it.

JavaScript's capabilities greatly depend on the environment it's running in. For instance, Node.js supports functions that allow JavaScript to read/write arbitrary files, perform network requests, etc.

In-browser JavaScript can do everything related to webpage manipulation, interaction with the user, and the webserver.

In-browser javascript can do,

- Add new HTML to the page, change the existing content, modify styles.
- React to user actions, run on mouse clicks, pointer movements, key presses.
- Send requests over the network to remote servers, download and upload files (so-called AJAX and COMET technologies).
- Get and set cookies, ask questions to the visitor, show messages.
- Remember the data on the client-side ("local storage").

What can't in-browser javascript do,

JavaScript's abilities in the browser are limited to protect the user's safety. The aim is to prevent an evil webpage from accessing private information or harming the user's data.

What makes javascript unique ?

There are at least *three* great things about JavaScript:

- Full integration with HTML/CSS.
- Simple things are done simply.
- Supported by all major browsers and enabled by default.

JavaScript is the only browser technology that combines these three things. This makes the javascript unique.

Languages over javascript:

- CoffeeScript is "syntactic sugar" for JavaScript. It introduces shorter syntax, allowing us to write clearer and more precise code. Usually, Ruby devs like it.
- TypeScript is concentrated on adding "strict data typing" to simplify the development and support of complex systems. It is developed by Microsoft.
- Flow also adds data typing, but in a different way. Developed by Facebook.
- Dart is a standalone language that has its own engine that runs in non-browser environments (like mobile apps), but also can be transpiled to JavaScript. Developed by Google.
- Brython is a Python transpiler to JavaScript that enables the writing of applications in pure Python without JavaScript.
- Kotlin is a modern, concise and safe programming language that can target the browser or Node.

Javascript fundamentals:

JavaScript programs can be inserted almost anywhere into an HTML document using the <script> tag.

```
<!DOCTYPE HTML>
<html>
<body>
  <p>Before the script...</p>
  <script>
    alert( 'Hello, world!' );
  </script>
  <p>...After the script.</p>
</body>
</html>
```

Modern markup:

The <script> tag has a few attributes that are rarely used nowadays but can still be found in old code:

The type attribute: <script type=…>

The old HTML standard, HTML4, required a script to have a type. Usually it was type="text/javascript". It's not required anymore. Also, the modern HTML standard totally changed the meaning of this attribute.

The language attribute: <script language=…>

This attribute was meant to show the language of the script. This attribute no longer makes sense because JavaScript is the default language. There is no need to use it.

External scripts:

If we have a lot of JavaScript code, we can put it into a separate file.

Script files are attached to HTML with the src attribute:

```
<script src="/path/to/script.js"></script>
```

To attach several scripts, use multiple tags:

```
<script src="/js/script1.js"></script>

<script src="/js/script2.js"></script> …
```

As a rule, only the simplest scripts are put into HTML. More complex ones reside in separate files.

The benefit of a separate file is that the browser will download it and store it in its cache.

Other pages that reference the same script will take it from the cache instead of downloading it, so the file is actually downloaded only once.

That reduces traffic and makes pages faster.

If src is set, the script content is ignored.

A single <script> tag can't have both the src attribute and code inside.

```
<script src="file.js">
  alert(1); // the content is ignored, because src is set
</script>
```

We must choose either an external <script src="…"> or a regular <script> with code.

```
<script src="file.js"></script>

<script>
  alert(1);
</script>
```

Code structure:

Statements - Statements are syntax constructs and commands that perform actions.

We've already seen a statement, alert('Hello, world!'), which shows the message "Hello, world!".

We can have as many statements in our code as we want. Statements can be separated with a semicolon.

alert('Hello'); alert('World');

Semicolons:

A semicolon may be omitted in most cases when a line break exists.

This would also work:

```
alert('Hello')

alert('World')
```

Here, JavaScript interprets the line break as an “implicit” semicolon. This is called an automatic semicolon insertion.

In most cases, a newline implies a semicolon. But “in most cases” does not mean “always”!

There are cases when a newline does not mean a semicolon. For example:

```
alert(3 +

1

+ 2);
```

The code outputs 6 because JavaScript does not insert semicolons here. It is intuitively obvious that if the line ends with a plus "+", then it is an “incomplete expression”, so a semicolon there would be incorrect. And in this case, that works as intended.

But there are situations where JavaScript “fails” to assume a semicolon where it is really needed.

Comments:

One-line comments start with two forward slash characters //.

Multiline comments start with a forward slash and an asterisk /* and end with an asterisk and a forward slash */.

There may not be /*...*/ inside another /*...*/.

Such code will die with an error:

```
/*

  /* nested comment ?!? */

*/

alert( 'World' );
```

Use strict:

The directive looks like a string: "use strict" or 'use strict'. When it is located at the top of a script, the whole script works the “modern” way.

For example:

```
"use strict";


// this code works the modern way
...
```

Quite soon we're going to learn functions (a way to group commands), so let's note in advance that "use strict" can be put at the beginning of a function. Doing that enables strict mode in that function only. But usually people use it for the whole script.

Please make sure that "use strict" is at the top of your scripts, otherwise strict mode may not be enabled.

Strict mode isn't enabled here:

```
alert("some code");
// "use strict" below is ignored--it must be at the top


"use strict";


// strict mode is not activated
```

Only comments may appear above "use strict".

Modern JavaScript supports "classes" and "modules" – advanced language structures (we'll surely get to them), that enable use strict automatically. So we don't need to add the "use strict" directive, if we use them.

So, for now "use strict"; is a welcome guest at the top of your scripts. Later, when your code is all in classes and modules, you may omit it.

Variables:

A variable is a "named storage" for data. We can use variables to store goodies, visitors, and other data.

To create a variable in JavaScript, use the let keyword.

```
let message;
```

var instead of let

In older scripts, you may also find another keyword: var instead of let:

```
var message = 'Hello';
```

The var keyword is *almost* the same as let. It also declares a variable but in a slightly different, "old-school" way.

A variable should be declared only once.

A repeated declaration of the same variable is an error:

```
let message = "This";


// repeated 'let' leads to an error
let message = "That"; // SyntaxError: 'message' has already been declared
```

So, we should declare a variable once and then refer to it without let.

Variable naming:

There are two limitations on variable names in JavaScript:

1. The name must contain only letters, digits, or the symbols $ and _.
2. The first character must not be a digit.

Examples of valid names:

```
let userName;
let test123;
```

When the name contains multiple words, camelCase is commonly used. That is: words go one after another, with each word except the first starting with a capital letter: myVeryLongName.

Examples of incorrect variable names:

```
let 1a; // cannot start with a digit
let my-name; // hyphens '-' aren't allowed in the name
```

Case matters

Variables named apple and APPLE are two different variables.

Non-Latin letters are allowed, but not recommended

It is possible to use any language, including Cyrillic letters, Chinese logograms and so on, like this:

```
let имя = '...';
let 我 = '...';
```

Technically, there is no error here. Such names are allowed, but there is an international convention to use English in variable names. Even if we're writing a small script, it may have a long life ahead. People from other countries may need to read it sometime.

There is a list of reserved words, which cannot be used as variable names because they are used by the language itself.

For example: let, class, return, and function are reserved.

The code below gives a syntax error:

```
let let = 5; // can't name a variable "let", error!
let return = 5; // also can't name it "return", error!
```

Normally, we need to define a variable before using it. But in the old times, it was technically possible to create a variable by a mere assignment of the value without using let. This still works now if we don't put use strict in our scripts to maintain compatibility with old scripts.

```
// note: no "use strict" in this example

num = 5; // the variable "num" is created if it didn't exist

alert(num); // 5
```

This is a bad practice and would cause an error in strict mode:

```
"use strict";

num = 5; // error: num is not defined
```

Constants:

To declare a constant (unchanging) variable, use const instead of let:

```
const myBirthday = '18.04.1982';
```

Variables declared using const are called "constants". They cannot be reassigned. An attempt to do so would cause an error:

```
const myBirthday = '18.04.1982';

myBirthday = '01.01.2001'; // error, can't reassign the constant!
```

Uppercase constants:

There is a widespread practice to use constants as aliases for difficult-to-remember values that are known before execution.

Such constants are named using capital letters and underscores.

Data types:

A value in JavaScript is always of a certain type. For example, a string or a number.

There are eight basic data types in JavaScript. Here, we'll cover them in general and in the next chapters we'll talk about each of them in detail.

We can put any type in a variable. For example, a variable can at one moment be a string and then store a number:

```
// no error

let message = "hello";

message = 123456;
```

Programming languages that allow such things, such as JavaScript, are called "dynamically typed", meaning that there exist data types, but variables are not bound to any of them.

Number:

The *number* type represents both integer and floating point numbers.

There are many operations for numbers, e.g. multiplication *, division /, addition +, subtraction -, and so on.

Besides regular numbers, there are so-called “special numeric values” which also belong to this data type: Infinity, -Infinity and NaN.

- Infinity represents the mathematical Infinity ∞. It is a special value that’s greater than any number.

We can get it as a result of division by zero:

```
alert( 1 / 0 ); // Infinity
```

Or just reference it directly:

```
alert( Infinity ); // Infinity
```

NaN represents a computational error. It is a result of an incorrect or an undefined mathematical operation, for instance:

```
alert( "not a number" / 2 ); // NaN, such division is erroneous
```

NaN is sticky. Any further mathematical operation on NaN returns NaN:

```
alert( NaN + 1 ); // NaN
alert( 3 * NaN ); // NaN
alert( "not a number" / 2 - 1 ); // NaN
```

So, if there’s a NaN somewhere in a mathematical expression, it propagates to the whole result (there’s only one exception to that: NaN ** 0 is 1).

Mathematical operations are safe

Doing maths is “safe” in JavaScript. We can do anything: divide by zero, treat non-numeric strings as numbers, etc.

The script will never stop with a fatal error (“die”). At worst, we’ll get NaN as the result.

BigInt

In JavaScript, the “number” type cannot safely represent integer values larger than

(2^{53}-1) (that’s 9007199254740991), or less than -(2^{53}-1) for negatives.

To be really precise, the “number” type can store larger integers (up to $1.7976931348623157 * 10^{308}$), but outside of the safe integer range $\pm(2^{53}-1)$ there’ll be a precision error, because not all digits fit into the fixed 64-bit storage. So an “approximate” value may be stored.

A BigInt value is created by appending n to the end of an integer:

```
// the "n" at the end means it's a BigInt

const bigInt = 1234567890123456789012345678901234567890n;
```

String:

A string in JavaScript must be surrounded by quotes.

```
let str = "Hello";

let str2 = 'Single quotes are ok too';

let phrase = `can embed another ${str}`;
```

In JavaScript, there are 3 types of quotes.

1. Double quotes: "Hello".
2. Single quotes: 'Hello'.
3. Backticks: `Hello`.